



BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

Research Paper

NAME: GROUP A

DATE: JUNE-AUGUST 2026

Title: Adaptive Level-of-Detail Terrain Rendering Using Camera-Distance-Based Geometry Selection

Abstract

This paper presents a camera-distance-based Adaptive Level of Detail (LOD) system developed as the novel contribution of Milestone 6 of a procedural terrain generation project. The system addresses the problem of rendering terrain with unnecessarily high geometric detail regardless of the viewer's distance from the terrain. Three levels of detail, High, Medium and Low, were implemented and automatically selected according to camera distance. High LOD is selected when the camera is close to the terrain, Medium LOD is selected at an intermediate distance and Low LOD is selected when the camera is far away. The Adaptive LOD system was integrated with the existing procedural terrain generation, rendering and animation components.

A fixed 60×60 terrain was used for the main experiment to isolate the effect of camera distance. At camera distances of 15, 30 and 60 units, the system selected High, Medium and Low LOD respectively. The corresponding geometry reductions were 0%, 75% and 93.75%. The results demonstrate that camera-distance-based Adaptive LOD can dynamically control terrain geometric complexity while maintaining higher detail for nearby terrain and reducing unnecessary detail for distant terrain.

Keywords

Adaptive LOD, Level of Detail, Procedural Terrain, Camera Distance, Geometry Reduction

Introduction

Procedural terrain generation is a technique used to automatically create terrain surfaces using mathematical functions and computational algorithms. Such systems can generate large environments containing thousands or millions of terrain points. While detailed terrain can improve visual quality, processing every terrain point at maximum resolution is not always necessary.

When terrain is viewed from a distance, small geometric details become less visually significant. Rendering these details can therefore result in unnecessary computational work. A level-of-detail technique addresses this problem by using different geometric resolutions depending on the importance of the terrain to the viewer.

Milestone 6 extends the existing procedural terrain generator by introducing camera-distance-based Adaptive Level of Detail as the novel system contribution. Instead of using one fixed terrain resolution, the proposed system automatically selects an appropriate LOD according to the distance between the camera and the terrain.

The proposed system uses three levels: High, Medium and Low. High LOD preserves the complete terrain resolution for nearby views, Medium LOD reduces the number of terrain points at intermediate distances and Low LOD uses a highly reduced representation for distant views.

The objective of this work is to develop and integrate a working Adaptive LOD system and evaluate its ability to reduce terrain geometric complexity as camera distance increases.

Methodology

Development Tools

The Adaptive LOD system was implemented using:

- Python – primary programming language.
- NumPy – numerical and array operations used by the terrain system.
- Matplotlib – visualization and interactive display of the terrain.
- Visual Studio Code – development environment.
- Custom Python modules – used to implement the LOD and Adaptive LOD algorithms.

Level of Detail System

The LOD system provides three levels of terrain resolution:

LOD	Level	Sampling Step	Description
High		1	Full terrain resolution
Medium		2	Reduced terrain resolution
Low		4	Highly reduced terrain resolution

For High LOD, $s=1$, so no terrain points are skipped.

For Medium LOD, $s=2$, so every second terrain point is retained.

For Low LOD, $s=4$, so every fourth terrain point is retained.

Adaptive LOD Selection and Geometry Reduction

The main novel contribution of this work is the automatic selection of terrain Level of Detail (LOD) based on camera distance. The system uses three levels: High, Medium and Low.

The selection function is:

$L(d) = \text{High}$, if $d < 20$

Medium, if $20 \leq d < 45$

Low, if $d \geq 45$

where **d** represents the camera distance from the terrain. The AdaptiveLOD class compares the current camera distance with these thresholds and selects the appropriate LOD level.

```
adaptive_lod.py • vertex.py terrain_generator.py X stochastic_analysis.py benchmark.py lighting.py mater
adaptive_lod.py
1 from lod import LOD
2 class AdaptiveLOD:
3
4     def __init__(self):
5
6         self.lod = LOD()
7
8     def select_level(self, camera_distance):
9
10        # Close camera = maximum detail
11        if camera_distance < 20:
12            return "high"
13
14        # Medium distance = medium detail
15        elif camera_distance < 45:
16            return "medium"
17
18        # Far distance = minimum detail
19        else:
20            return "low"
21    def generate(self, terrain, camera_distance):
22
23        level = self.select_level(
24            camera_distance
25        )
26
27        reduced = self.lod.reduce(
28            terrain,
29            level
30        )
31        return reduced, level
```

After selecting the LOD level, the terrain is reduced using a sampling step. High LOD uses a step of 1, Medium uses 2 and Low uses 4.

```
stochastic_analysis.py benchmark.py lighting.py material.py renderer.py visualizer.py lod.py X
lod.py
1 class LOD:
2
3     def __init__(self):
4
5         self.levels = {
6             "high": 1,
7             "medium": 2,
8             "low": 4
9         }
10
11    def reduce(self, terrain, level="high"):
12
13        step = self.levels[level]
14
15        return [
16            row[::step]
17            for row in terrain[::step]
18        ]
```

The sampling operation retains terrain points according to the selected step. For a 60×60 terrain, High LOD retains all 3,600 points, Medium LOD reduces the terrain to 900 points and Low LOD reduces it to 225 points.

System Integration

The Adaptive LOD system was integrated into the existing procedural terrain generation and animation system.

During each animation frame, the current camera distance is calculated. This distance is passed to the AdaptiveLOD module, which determines the appropriate level of detail.

The selected LOD is then passed to the geometry reduction module. The reduced terrain is subsequently converted into coordinate arrays and sent to the existing rendering system.

The complete processing pipeline is:

Camera Distance → Adaptive LOD Selection → Geometry Reduction → Terrain Rendering

This integration allows the terrain resolution to change dynamically while the system is running. The Adaptive LOD system therefore does not operate as an independent demonstration. It forms part of the final terrain system and works together with the existing terrain generation, rendering, lighting, water animation and camera animation components.

Experimental Testing

A fixed 60×60 terrain was selected for the main Adaptive LOD experiment. Keeping the terrain size constant allowed the effect of camera distance to be isolated.

Three camera distances were tested:

- 15 units representing a nearby camera
- 30 units representing an intermediate camera
- 60 units representing a distant camera

These distances were selected to trigger the three different LOD levels.

For each test, the following values were recorded:

1. Camera distance
2. Selected LOD
3. Sampling step
4. Original number of terrain points
5. Number of points after LOD reduction
6. Percentage of geometry reduction

The geometry reduction was calculated using:

$$\text{Reduction (\%)} = ((\text{Original Points} - \text{LOD Points}) / \text{Original Points}) \times 100$$

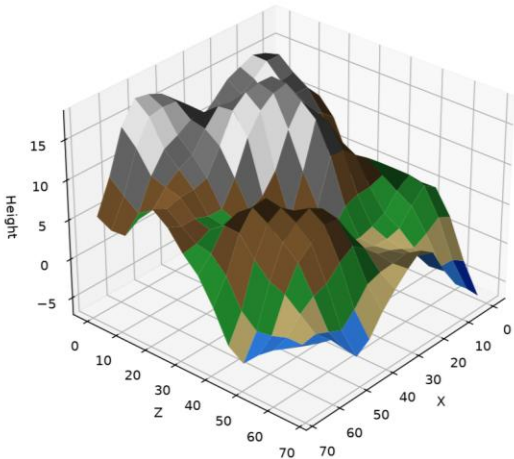
Experimental Results

The system was tested using a fixed 60 × 60 terrain at three different camera distances.

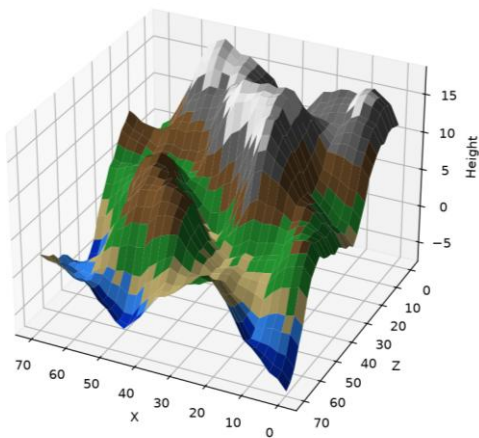
Terrain Size	Camera Distance	Selected LOD	Original Points	LOD Points	Reduction
60 × 60	16.1	High	3600	3600	0%
60 × 60	25.7	Medium	3600	900	75%
60 × 60	58.9	Low	3600	225	93.75%

Table 1: Adaptive LOD Experimental Results

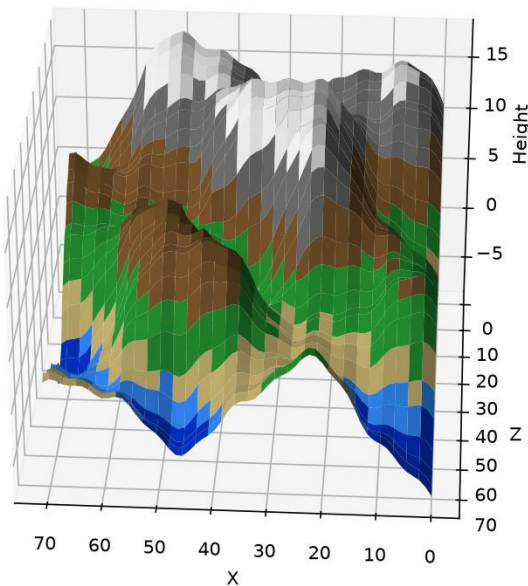
Procedural Terrain - Adaptive LOD: LOW | Distance: 58.9 | Points: 225



Procedural Terrain - Adaptive LOD: MEDIUM | Distance: 25.7 | Points: 900



Procedural Terrain - Adaptive LOD: HIGH | Distance: 16.1 | Points: 3600



Discussion

The results demonstrate that the Adaptive LOD system successfully changes terrain resolution according to camera distance.

At 15 units, High LOD retained all 3,600 terrain points, preserving maximum detail for nearby terrain.

At 30 units, Medium LOD reduced the terrain to 900 points, achieving a 75% geometry reduction.

At 60 units, Low LOD reduced the terrain to 225 points, achieving a 93.75% reduction.

These results demonstrate that terrain complexity decreases as the camera moves farther away.

The system therefore provides higher detail when it is visually important and reduces unnecessary geometric detail when the terrain is distant.

The experiment also confirms that LOD selection is controlled by camera distance rather than camera rotation. However, geometry reduction does not necessarily represent the same percentage improvement in rendering time. Further frame-time profiling would be required to measure the exact performance improvement.

Conclusion

Milestone 6 introduced camera-distance-based Adaptive Level of Detail as a novel system contribution to the procedural terrain generator. The system automatically selects High, Medium or Low LOD according to camera distance. Using a fixed 60×60 terrain, the system achieved 75% geometry reduction at Medium LOD and 93.75% at Low LOD. The results demonstrate that Adaptive LOD can reduce unnecessary terrain geometry while maintaining higher detail for nearby views.

References

- [1] D. Luebke, M. Reddy, J. D. Cohen, A. Varshney, B. Watson and R. Huebner, *Level of Detail for 3D Graphics*. Morgan Kaufmann, 2002.
- [2] J. D. Foley, A. van Dam, S. K. Feiner and J. F. Hughes, *Computer Graphics: Principles and Practice*. Addison-Wesley.
- [3] Matplotlib Development Team, *Matplotlib Documentation*.